3.1 Modularizacion en C++, Java, C, Js y python
se expone que la modularizacion permite descomponer problemas mediante el uso de funciones (que realizan acciones y devuelven un resultado) y procedimiento (que realizan tareas especificas sin devolver valor) operando ambos a traves de parametros

3.2 Procedimiento y funciones necesarias del proyecto
El sistema se divide estructuralmente en dos bloques logicos procesos de punto de venta y funciones de apoyo"

3.2.1 Funciones de apoyo
se agrupan en categorias especializadas (menú, entrada y validacion formateo y fecha/hora) para simplificar la interaccion y garantizar la integridad de los datos, separados estas utilidades de la logica del negocio

3.3 Funciones de menú
proporciona una interfaz estructurada y accesible para que los usuarios naveguen por el programa

3.3.1 Funcion mostrar menú
genera una representacion visual en consola concatenando en arreglo de cadenas de texto, para su recorrido, se implementa el "ciclo for mejorado" (for-each), que simplifica la iteracion de los arreglos de manera segura

3.4 Las funciones de entrada y validacion
establecen el canal de comunicacion y aseguran que los datos recibidos sean los correctos

3.4.1 Funcion dialogo
solicita y lee directamente una cadena de texto ingresada por el teclado su usuario

3.4.2 Función imprimir menú
combinar el título del menú con las opciones disponibles
y capturar la elección del usuario en una sola función

3.5 Función leer
lee la entrada del usuario, valida que no sea nula y elimina los espacios en blanco al principio y al final de la cadena

3.5.1 Función evaluar numérico entero
verifica si una cadena representa un número entero iterando sobre ella para comprobar que cada carácter sea un dígito válido

3.5.2 Función evaluar numérico double, determina si la cadena es un decimal válido comprobando que solo contenga dígitos, y como máximo un solo punto decimal

3.5.3 Función evaluar numérico
Actua como un selector: recibe un parámetro para decidir que tipo de validación aplicar (entero o decimal) utilizando las funciones previas

3.6 Funciones de formateo
permiten dar un formato de salida uniforme a los enteros para mejor la visualización de los datos

3.6.1 Función rellenar espacios
añade espacios a una cadena hasta completar un tamaño dado empleando funciones nativas de cada lenguaje como: std::setw, string.format, pad end o (just)

3.6.2 Funcion id ticket siguiente
genera el número correlativo del proximo ticket convirtiendo el actual a entero, sumandole 1 y volviendolo a formatear como texto con ceros a la izquierda

3.6.3 Funcion obtener ultima posicion
recorre las filas de un arreglo bidimensional verificando su primera columna para identificar la ultima posicion vacia, permitiendo insertar datos sin sobrescribir información

3.7 Funcion fecha
obtiene la fecha del sistema y extrae el dia, mes y año para devolverlos formateados como "dd-mm-yyyy"

3.8 Resumen del capitulo
concluye como la descomposicion arquitectonica en modulos logicos (menus, validaciones, repeticiones) optimiza la legibilidad y resta las bases para el trabajo colaborativo

3.8.1 Aplicacion del pensamiento critico
se anticiparon escenarios de error, cuestionando como el sistema debe reaccionar ante datos anomalos o comportamientos inesperados para evitar colapsos

3.8.2 Aplicacion del pensamiento de diseño
se hizo una experiencia de uso amigable y a prueba de fallos mediante menus estructurados para la consola de texto

3.8.3 Aplicación del pensamiento computacional
Se abstraerán los procesos repetitivos en plantillas lógicas e instrucciones para facilitar su realización en el proyecto

3.9 Actividades

Ejercicios propuestos para reforzar y aplicar el código desarrollado

3.9.1 Actividades de pensamiento crítico
Análisis del propósito, robustez y eficiencia lógica de cada procedimiento evaluado

3.9.2 Actividades de pensamiento computacional
Evaluación de la abstracción descomposición y reconocimiento de patrones en las funciones

3.9.3 Actividades de pensamientos de diseño
Revisión de la usabilidad, cohesión y reutilización del código desde la perspectiva de otro programador

3.9.4 Actividades de código limpio
se cambiaron variables con nombres genéricos como opt, aux, cat o bandera por identificadores semánticos y precisos (como opcion seleccionada, texto a formatear o es_vacio)

3.9.5 Actividades de pensamiento de diseño
análisis reflexivo sobre usabilidad, cohesión y reutilización para los programadores

3.9.6 Actividad de una sola función de validación numérica

Ejercicio para consolidar las validaciones numéricas en una función única y más flexible que acepte indicadores descriptivos como "entero" o "decimal"

3.9.7 Actividades una mejor función llamada intervalo dato recto para crear una función integral que incluya un "bucle de reintento" atrapando al usuario hasta que ingrese un dato correcto para evitar fallos

Capitulo3.java | Ejercicio01_Figura29_FuncionSuma.java | Ejercicio02_Figura30_ProcedimientoImprimir.java

```java
package a2261330036_estudiodecaso;

/**
 * Ejercicio 01 - Figura 29: Estructura de una funcion
 * Descripcion: Demuestra la declaracion y llamado de una funcion que recibe
 * dos parametros enteros y retorna la suma de ambos.
 */
public class Ejercicio01_Figura29_FuncionSuma {

    // Codigo en Java de la Figura 29
    public static int sumar(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 01: Figura 29 - Estructura de una funcion ===");
        int num1 = 15;
        int num2 = 25;
        int resultado = sumar(num1, num2);
        System.out.println("La suma de " + num1 + " + " + num2 + " es: " + resultado);
    }
}
```

Console | Java Developers 2026-09 Release

<terminated> Ejercicio01_Figura29_FuncionSuma [Java Application] C:\Program Files\Java\jdk-26.0.2.

```
=== Ejercicio 01: Figura 29 - Estructura de una funcion ===
La suma de 15 + 25 es: 40
```

Capitulo3.java | Ejercicio01_Figura29_FuncionSuma.java | Ejercicio02_Figura30_ProcedimientoImprimir.java

```java
package a2261330036_estudiodecaso;

/**
 * Ejercicio 02 - Figura 30: Estructura de un procedimiento
 * Descripcion: Demuestra un procedimiento de tipo void que realiza una accion
 * (imprimir un texto en consola) sin devolver ningun valor.
 */
public class Ejercicio02_Figura30_ProcedimientoImprimir {

    // Codigo en Java de la Figura 30
    public static void ImprimirNombre(String parametro) {
        System.out.println(parametro);
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 02: Figura 30 - Estructura de un procedimiento ===");
        ImprimirNombre("Hola mundo desde un procedimiento modularizado en Java.");
        ImprimirNombre("Este metodo ejecuta una accion y no retorna valor.");
    }
}
```

Console | Java Developers 2026-09 Release

<terminated> Ejercicio02_Figura30_ProcedimientoImprimir [Java Application] C:\Program Files\Java\j

```
=== Ejercicio 02: Figura 30 - Estructura de un procedimiento ===
Hola mundo desde un procedimiento modularizado en Java.
Este metodo ejecuta una accion y no retorna valor.
```

Ejercicio01_Figura29_FuncionSum... | Ejercicio02_Figura30_Procedimie... | Ejercicio03_3_3_1_MostrarMenu...

```java
package a2261330036_estudiodecaso;

/**
 * Ejercicio 03 - Seccion 3.3.1: Funcion MostrarMenu con ciclo While (Pagina 64)
 * Descripcion: Genera una representacion visual de un menu de opciones
 * a partir de un arreglo de cadenas usando un bucle while e indexacion.
 */
public class Ejercicio03_3_3_1_MostrarMenuWhile {

    // Codigo en Java de la Pagina 64
    public static String MostrarMenu(String[] opciones) {
        String cadena = "";
        int i = 0;
        while (i < opciones.length) {
            cadena += (i + 1) + ". " + opciones[i] + "\n";
            i++;
        }
        return cadena;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 03: Seccion 3.3.1 - MostrarMenu con While ===");
        String[] opcionesMenu = {
            "Productos",
            "Punto de Venta",
            "Inventario",
            "Listado de Ventas",
            "Salida"
        };

        String menuGenerado = MostrarMenu(opcionesMenu);
        System.out.println("Menu generado:\n" + menuGenerado);
    }
}
```

Console | Java Developers 2026-09 Release

<terminated> Ejercicio03_3_3_1_MostrarMenuWhile [Java Application] C:\Program Files\Java\jdk-26.0.

```
=== Ejercicio 03: Seccion 3.3.1 - MostrarMenu con While ===
Menu generado:
1. Productos
2. Punto de Venta
3. Inventario
4. Listado de Ventas
5. Salida
```

```java
package a2261330036_estudiodecaso;

/**
 * Ejercicio 04 - Seccion 3.3.1: Funcion MostrarMenu con ciclo For-Each (Pagina 66)
 * Descripcion: Recorre el arreglo de opciones mediante el ciclo for mejorado
 * evitando la necesidad de manejar indices manualmente.
 */
public class Ejercicio04_3_3_1_MostrarMenuForEach {

    // Codigo en Java de la Pagina 66
    public static String MostrarMenu(String[] opciones) {
        String cadena = "";
        for (String info : opciones) {
            cadena = cadena + info + "\n";
        }
        return cadena;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 04: Seccion 3.3.1 - MostrarMenu con For-Each ===");
        String[] listaOpciones = {
            "1.- Modificar Producto",
            "2.- Listado de Productos",
            "3.- Salida"
        };

        String resultado = MostrarMenu(listaOpciones);
        System.out.println("Menu generado con bucle for-each:\n" + resultado);
    }
}
```

```
=== Ejercicio 04: Seccion 3.3.1 - MostrarMenu con For-Each ===
Menu generado con bucle for-each:
1.- Modificar Producto
2.- Listado de Productos
3.- Salida
```

```java
package a2261330036_estudiodecaso;

import java.io.BufferedReader;

/**
 * Ejercicio 05 - Seccion 3.4.1: Funcion Dialogo (Pagina 68)
 * Descripcion: Solicita una entrada al usuario mostrando un mensaje o etiqueta
 * y retorna la linea de texto introducida mediante teclado.
 */
public class Ejercicio05_3_4_1_FuncionDialogo {

    // Codigo en Java de la Pagina 68
    public static String Dialogo(String texto) throws IOException {
        String cadena;
        System.out.println(texto + " : ");
        BufferedReader lectura = new BufferedReader(new InputStreamReader(System.in));
        cadena = lectura.readLine();
        return cadena;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 05: Seccion 3.4.1 - Funcion Dialogo ===");
        try {
            String respuesta = Dialogo("Introduce el nombre del producto");
            System.out.println("Texto capturado por la funcion Dialogo: " + respuesta);
        } catch (IOException e) {
            System.err.println("Error de lectura: " + e.getMessage());
        }
    }
}
```

```
=== Ejercicio 04: Seccion 3.3.1 - MostrarMenu con For-Each ===
Menu generado con bucle for-each:
1.- Modificar Producto
2.- Listado de Productos
3.- Salida
```

```java
package a2261330036_estudiodecaso;

import java.io.BufferedReader;

/**
 * Ejercicio 06 - Seccion 3.4.2: Funcion DesplegarMenu (Pagina 71)
 * Descripcion: Combina el titulo, las opciones formateadas con MostrarMenu
 * y solicita al usuario que elija una opcion usando Dialogo.
 */
public class Ejercicio06_3_4_2_DesplegarMenu {

    // Funcion MostrarMenu auxiliar
    public static String MostrarMenu(String[] opciones) {
        String cadena = "";
        int i = 0;
        while (i < opciones.length) {
            cadena += (i + 1) + ". - " + opciones[i] + "\n";
            i++;
        }
        return cadena;
    }

    // Funcion Dialogo auxiliar
    public static String Dialogo(String texto) throws IOException {
        String cadena;
        System.out.println(texto + " : ");
        BufferedReader lectura = new BufferedReader(new InputStreamReader(System.in));
        cadena = lectura.readLine();
        return cadena;
    }

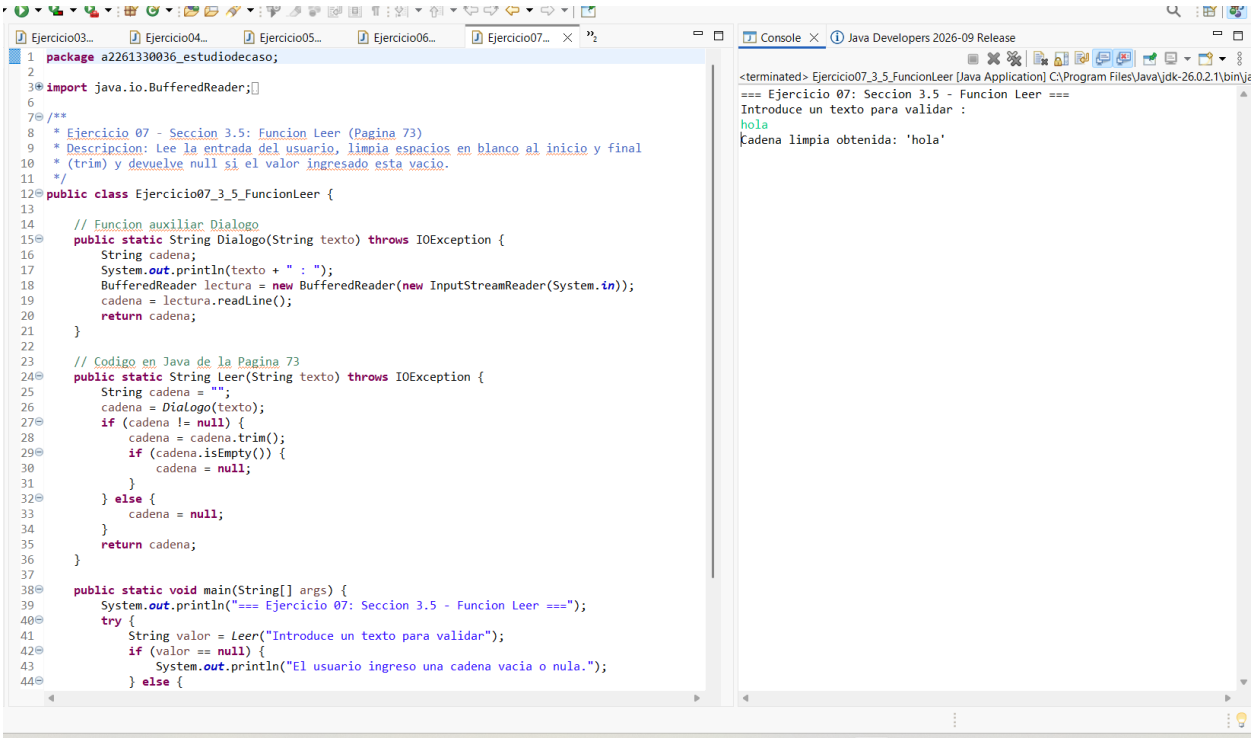
    // Codigo en Java de la Pagina 71
    public static String DesplegarMenu(String Titulo1, String[] menu) throws IOException {
        String cadena;
        cadena = Titulo1 + "\n" + "\n";
        cadena = cadena + MostrarMenu(menu);
        cadena = cadena + "\n Que opcion deseas ";
        return cadena = Dialogo(cadena);
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 06: Seccion 3.4.2 - DesplegarMenu ===");
```

```
=== Ejercicio 06: Seccion 3.4.2 - DesplegarMenu ===
Menu de Punto de Tienda de Abarrotes la Pequeña

1. - Productos
2. - Punto de Venta
3. - Inventario
4. - Listado de Ventas
5. - Salida

 Que opcion deseas  :
3
Opcion seleccionada por el usuario: 3
```



```java
package a2261330036_estudiodecaso;

import java.io.BufferedReader;

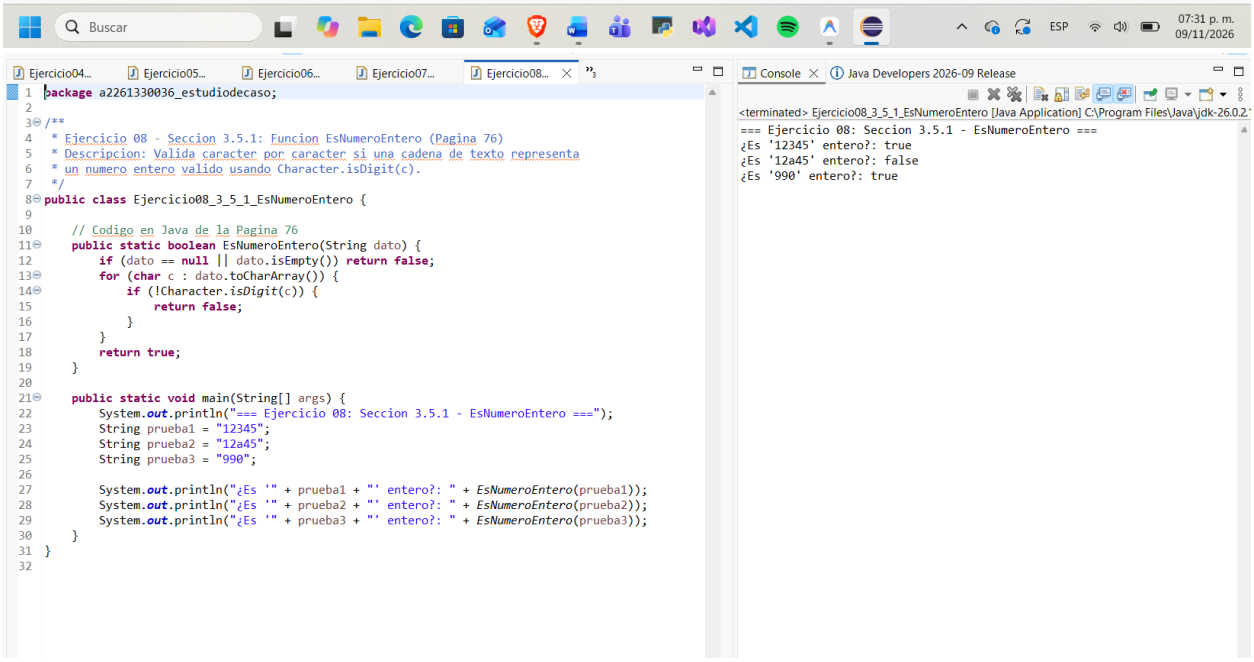
/**
 * Ejercicio 07 - Seccion 3.5: Funcion Leer (Pagina 73)
 * Descripcion: Lee la entrada del usuario, limpia espacios en blanco al inicio y final
 * (trim) y devuelve null si el valor ingresado esta vacio.
 */
public class Ejercicio07_3_5_FuncionLeer {

    // Funcion auxiliar Dialogo
    public static String Dialogo(String texto) throws IOException {
        String cadena;
        System.out.println(texto + " : ");
        BufferedReader lectura = new BufferedReader(new InputStreamReader(System.in));
        cadena = lectura.readLine();
        return cadena;
    }

    // Codigo en Java de la Pagina 73
    public static String Leer(String texto) throws IOException {
        String cadena = "";
        cadena = DiaLogo(texto);
        if (cadena != null) {
            cadena = cadena.trim();
            if (cadena.isEmpty()) {
                cadena = null;
            }
        } else {
            cadena = null;
        }
        return cadena;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 07: Seccion 3.5 - Funcion Leer ===");
        try {
            String valor = Leer("Introduce un texto para validar");
            if (valor == null) {
                System.out.println("El usuario ingreso una cadena vacia o nula.");
            } else {
```

```
=== Ejercicio 07: Seccion 3.5 - Funcion Leer ===
Introduce un texto para validar :
hola
Cadena limpia obtenida: 'hola'
```



```java
package a2261330036_estudiodecaso;

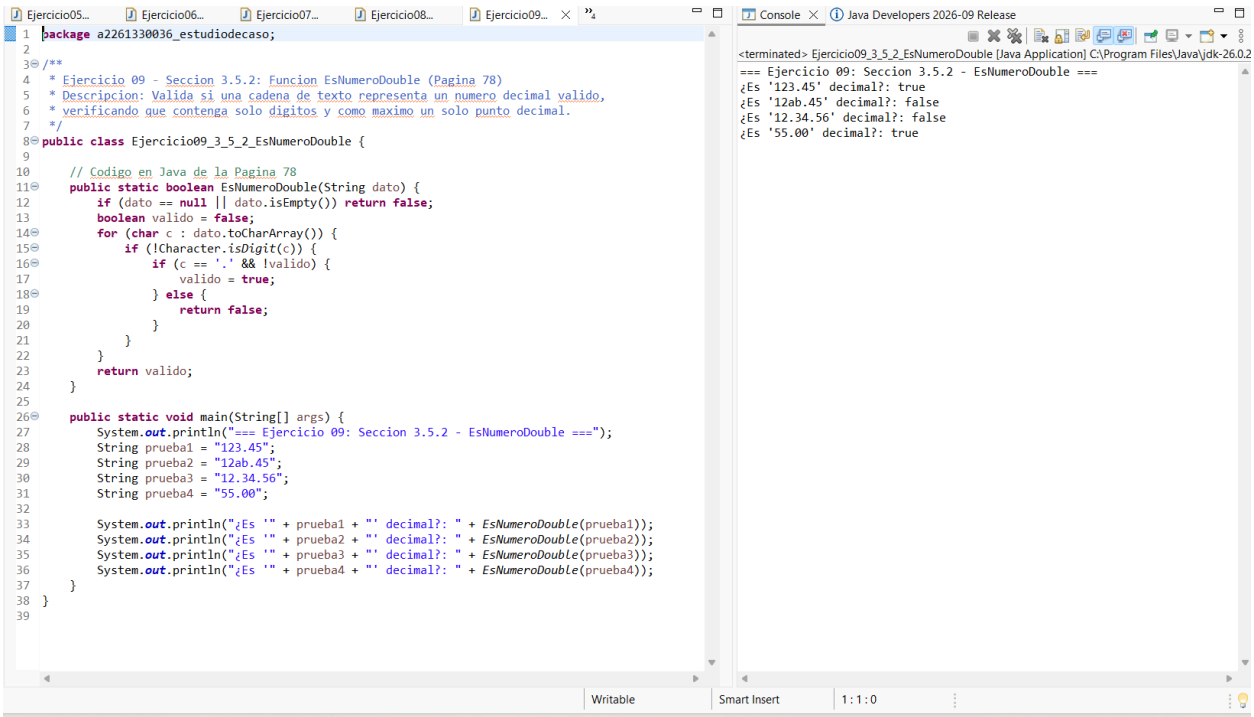
/**
 * Ejercicio 08 - Seccion 3.5.1: Funcion EsNumeroEntero (Pagina 76)
 * Descripcion: Valida caracter por caracter si una cadena de texto representa
 * un numero entero valido usando Character.isDigit(c).
 */
public class Ejercicio08_3_5_1_EsNumeroEntero {

    // Codigo en Java de la Pagina 76
    public static boolean EsNumeroEntero(String dato) {
        if (dato == null || dato.isEmpty()) return false;
        for (char c : dato.toCharArray()) {
            if (!Character.isDigit(c)) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 08: Seccion 3.5.1 - EsNumeroEntero ===");
        String prueba1 = "12345";
        String prueba2 = "12a45";
        String prueba3 = "990";

        System.out.println("¿Es '" + prueba1 + "' entero?: " + EsNumeroEntero(prueba1));
        System.out.println("¿Es '" + prueba2 + "' entero?: " + EsNumeroEntero(prueba2));
        System.out.println("¿Es '" + prueba3 + "' entero?: " + EsNumeroEntero(prueba3));
    }
}
```

```
=== Ejercicio 08: Seccion 3.5.1 - EsNumeroEntero ===
¿Es '12345' entero?: true
¿Es '12a45' entero?: false
¿Es '990' entero?: true
```



```java
package a2261330036_estudiodecaso;

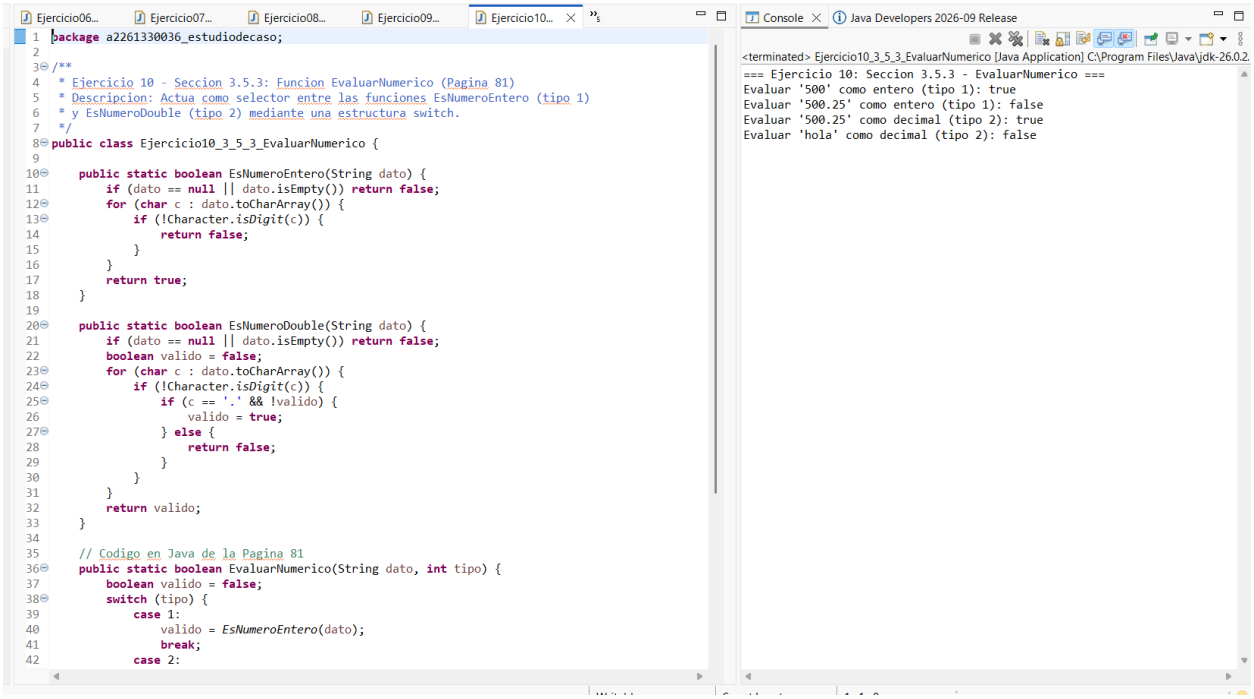
/**
 * Ejercicio 09 - Seccion 3.5.2: Funcion EsNumeroDouble (Pagina 78)
 * Descripcion: Valida si una cadena de texto representa un numero decimal valido,
 * verificando que contenga solo digitos y como maximo un solo punto decimal.
 */
public class Ejercicio09_3_5_2_EsNumeroDouble {

    // Codigo en Java de la Pagina 78
    public static boolean EsNumeroDouble(String dato) {
        if (dato == null || dato.isEmpty()) return false;
        boolean valido = false;
        for (char c : dato.toCharArray()) {
            if (!Character.isDigit(c)) {
                if (c == '.' && !valido) {
                    valido = true;
                } else {
                    return false;
                }
            }
        }
        return valido;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 09: Seccion 3.5.2 - EsNumeroDouble ===");
        String prueba1 = "123.45";
        String prueba2 = "12ab.45";
        String prueba3 = "12.34.56";
        String prueba4 = "55.00";

        System.out.println("¿Es '" + prueba1 + "' decimal?: " + EsNumeroDouble(prueba1));
        System.out.println("¿Es '" + prueba2 + "' decimal?: " + EsNumeroDouble(prueba2));
        System.out.println("¿Es '" + prueba3 + "' decimal?: " + EsNumeroDouble(prueba3));
        System.out.println("¿Es '" + prueba4 + "' decimal?: " + EsNumeroDouble(prueba4));
    }
}
```

```
<terminated> Ejercicio09_3_5_2_EsNumeroDouble [Java Application] C:\Program Files\Java\jdk-26.0.2
=== Ejercicio 09: Seccion 3.5.2 - EsNumeroDouble ===
¿Es '123.45' decimal?: true
¿Es '12ab.45' decimal?: false
¿Es '12.34.56' decimal?: false
¿Es '55.00' decimal?: true
```



```java
package a2261330036_estudiodecaso;

/**
 * Ejercicio 10 - Seccion 3.5.3: Funcion EvaluarNumerico (Pagina 81)
 * Descripcion: Actua como selector entre las funciones EsNumeroEntero (tipo 1)
 * y EsNumeroDouble (tipo 2) mediante una estructura switch.
 */
public class Ejercicio10_3_5_3_EvaluarNumerico {

    public static boolean EsNumeroEntero(String dato) {
        if (dato == null || dato.isEmpty()) return false;
        for (char c : dato.toCharArray()) {
            if (!Character.isDigit(c)) {
                return false;
            }
        }
        return true;
    }

    public static boolean EsNumeroDouble(String dato) {
        if (dato == null || dato.isEmpty()) return false;
        boolean valido = false;
        for (char c : dato.toCharArray()) {
            if (!Character.isDigit(c)) {
                if (c == '.' && !valido) {
                    valido = true;
                } else {
                    return false;
                }
            }
        }
        return valido;
    }

    // Codigo en Java de la Pagina 81
    public static boolean EvaluarNumerico(String dato, int tipo) {
        boolean valido = false;
        switch (tipo) {
            case 1:
                valido = EsNumeroEntero(dato);
                break;
            case 2:
```

```
<terminated> Ejercicio10_3_5_3_EvaluarNumerico [Java Application] C:\Program Files\Java\jdk-26.0.2
=== Ejercicio 10: Seccion 3.5.3 - EvaluarNumerico ===
Evaluar '500' como entero (tipo 1): true
Evaluar '500.25' como entero (tipo 1): false
Evaluar '500.25' como decimal (tipo 2): true
Evaluar 'hola' como decimal (tipo 2): false
```

Ejercicio07... Ejercicio08... Ejercicio09... Ejercicio10... Ejercicio11...

```java
package a2261330036_estudiodecaso;

/**
 * Ejercicio 11 - Seccion 3.6.1: Funcion RellenarEspacios con bucle For (Pagina 83)
 * Descripcion: Anade espacios en blanco al final de una cadena hasta completar
 * el tamano especificado mediante un ciclo for manual.
 */
public class Ejercicio11_3_6_1_RellenarEspacios {

    // Codigo en Java de la Pagina 83
    public static String RellenarEspacios(String dato, int tamano) {
        String cadena;
        cadena = dato;
        for (int ciclo = dato.length() - 1; ciclo < tamano; ciclo++) {
            cadena = cadena + " ";
        }
        return cadena;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 11: Seccion 3.6.1 - RellenarEspacios con Bucle ===");
        String producto = "Arroz";
        String alineado = RellenarEspacios(producto, 15);
        System.out.println("Original: '" + producto + "'");
        System.out.println("Alineado: '" + alineado + "' (longitud final: " + alineado.length()
    }
}
```

Console — Java Developers 2026-09 Release

```
<terminated> Ejercicio11_3_6_1_RellenarEspacios [Java Application] C:\Program Files\Java\jdk-26.0.2
=== Ejercicio 11: Seccion 3.6.1 - RellenarEspacios con Bucle ===
Original: 'Arroz'
Alineado: 'Arroz           ' (longitud final: 16)
```

Ejercicio08... Ejercicio09... Ejercicio10... Ejercicio11... Ejercicio12...

```java
package a2261330036_estudiodecaso;

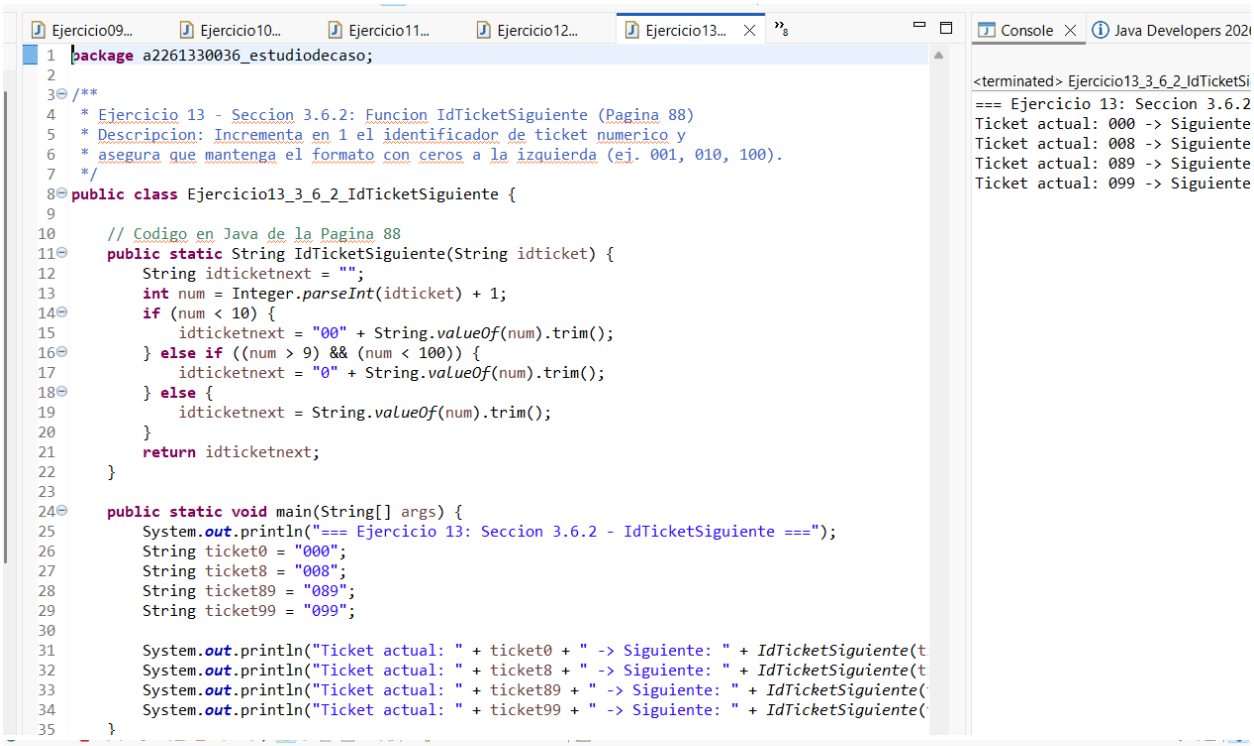
/**
 * Ejercicio 12 - Seccion 3.6.1: Funcion RellenarEspacios usando String.format (Pagina 85)
 * Descripcion: Utiliza las utilidades nativas de formato de Java (String.format con %1$-Ns)
 * para alinear a la izquierda rellenando con espacios hasta el ancho especificado.
 */
public class Ejercicio12_3_6_1_RellenarEspaciosFormato {

    // Codigo en Java de la Pagina 85
    public static String Rellenarespacios(String dato, int tamano) {
        return String.format("%1$-" + tamano + "s", dato);
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 12: Seccion 3.6.1 - RellenarEspacios con String.format
        String texto = "Aceite 1L";
        String resultado = Rellenarespacios(texto, 20);
        System.out.println("Texto formateado: '" + resultado + "' (longitud: " + resultado.lengt
    }
}
```

Console — Java Developers 2026-09 Release

```
<terminated> Ejercicio12_3_6_1_RellenarEspaciosFormato [Java Application] C:\Program Files\Java\jdk
=== Ejercicio 12: Seccion 3.6.1 - RellenarEspacios con String.format ===
Texto formateado: 'Aceite 1L           ' (longitud: 20)
```



```java
package a2261330036_estudiodecaso;

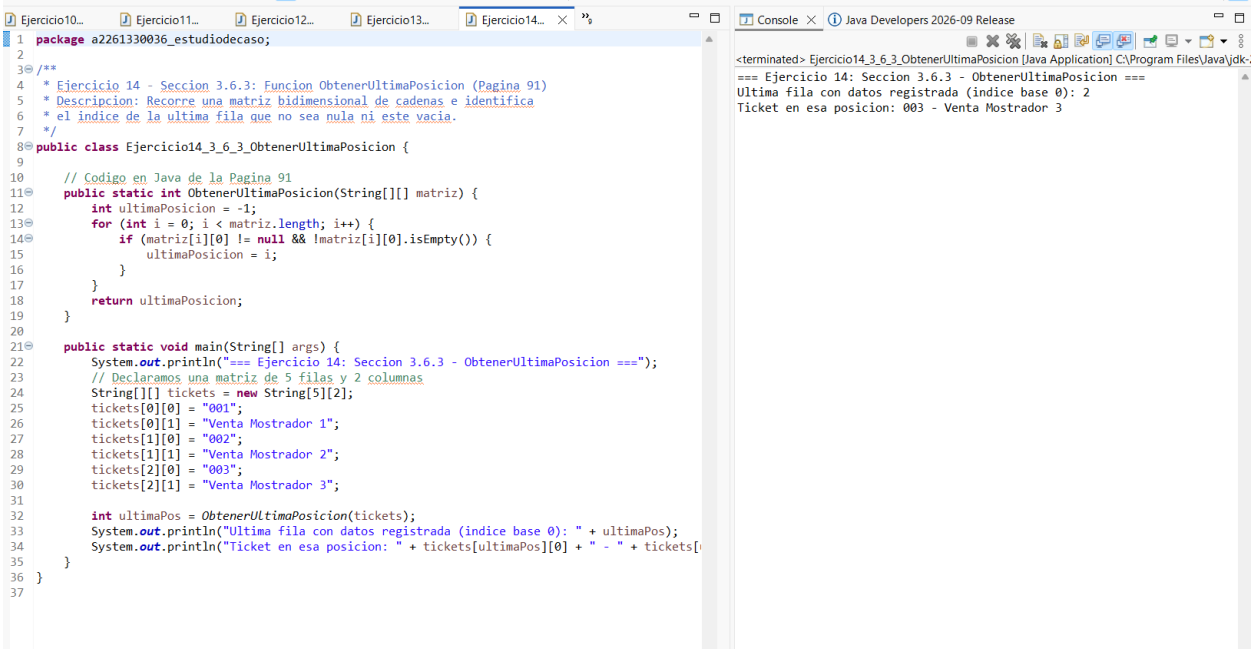
/**
 * Ejercicio 13 - Seccion 3.6.2: Funcion IdTicketSiguiente (Pagina 88)
 * Descripcion: Incrementa en 1 el identificador de ticket numerico y
 * asegura que mantenga el formato con ceros a la izquierda (ej. 001, 010, 100).
 */
public class Ejercicio13_3_6_2_IdTicketSiguiente {

    // Codigo en Java de la Pagina 88
    public static String IdTicketSiguiente(String idticket) {
        String idticketnext = "";
        int num = Integer.parseInt(idticket) + 1;
        if (num < 10) {
            idticketnext = "00" + String.valueOf(num).trim();
        } else if ((num > 9) && (num < 100)) {
            idticketnext = "0" + String.valueOf(num).trim();
        } else {
            idticketnext = String.valueOf(num).trim();
        }
        return idticketnext;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 13: Seccion 3.6.2 - IdTicketSiguiente ===");
        String ticket0 = "000";
        String ticket8 = "008";
        String ticket89 = "089";
        String ticket99 = "099";

        System.out.println("Ticket actual: " + ticket0 + " -> Siguiente: " + IdTicketSiguiente(t
        System.out.println("Ticket actual: " + ticket8 + " -> Siguiente: " + IdTicketSiguiente(t
        System.out.println("Ticket actual: " + ticket89 + " -> Siguiente: " + IdTicketSiguiente(
        System.out.println("Ticket actual: " + ticket99 + " -> Siguiente: " + IdTicketSiguiente(
    }
```

```
<terminated> Ejercicio13_3_6_2_IdTicketSi
=== Ejercicio 13: Seccion 3.6.2
Ticket actual: 000 -> Siguiente
Ticket actual: 008 -> Siguiente
Ticket actual: 089 -> Siguiente
Ticket actual: 099 -> Siguiente
```



```java
package a2261330036_estudiodecaso;

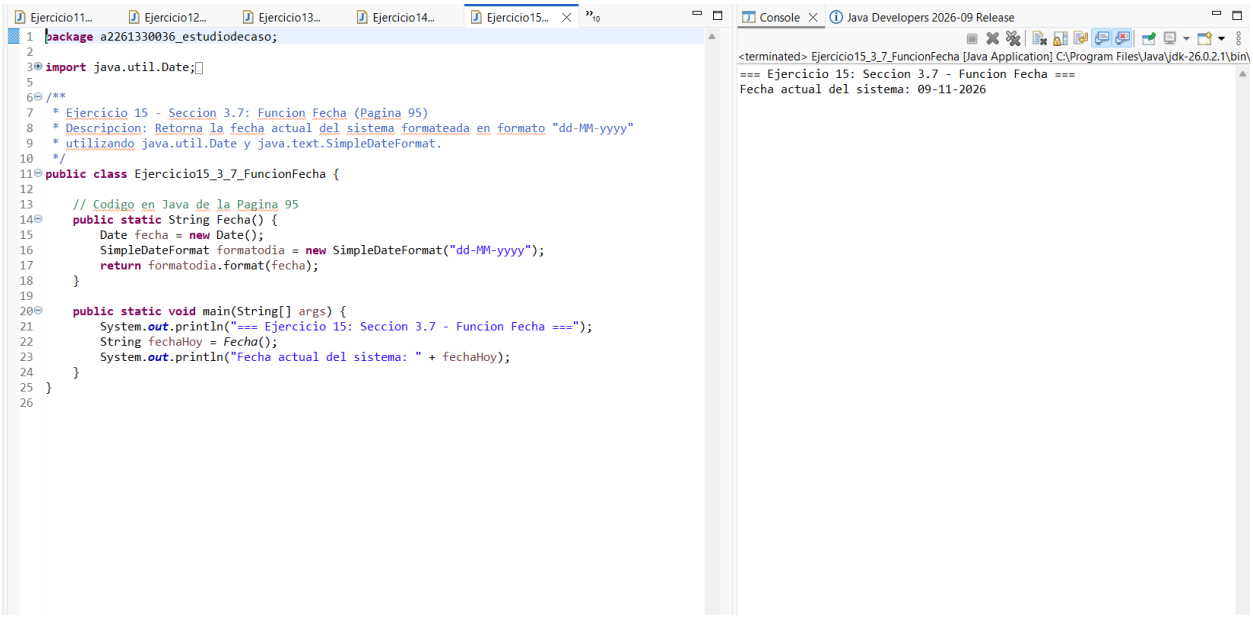
/**
 * Ejercicio 14 - Seccion 3.6.3: Funcion ObtenerUltimaPosicion (Pagina 91)
 * Descripcion: Recorre una matriz bidimensional de cadenas e identifica
 * el indice de la ultima fila que no sea nula ni este vacia.
 */
public class Ejercicio14_3_6_3_ObtenerUltimaPosicion {

    // Codigo en Java de la Pagina 91
    public static int ObtenerUltimaPosicion(String[][] matriz) {
        int ultimaPosicion = -1;
        for (int i = 0; i < matriz.length; i++) {
            if (matriz[i][0] != null && !matriz[i][0].isEmpty()) {
                ultimaPosicion = i;
            }
        }
        return ultimaPosicion;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 14: Seccion 3.6.3 - ObtenerUltimaPosicion ===");
        // Declaramos una matriz de 5 filas y 2 columnas
        String[][] tickets = new String[5][2];
        tickets[0][0] = "001";
        tickets[0][1] = "Venta Mostrador 1";
        tickets[1][0] = "002";
        tickets[1][1] = "Venta Mostrador 2";
        tickets[2][0] = "003";
        tickets[2][1] = "Venta Mostrador 3";

        int ultimaPos = ObtenerUltimaPosicion(tickets);
        System.out.println("Ultima fila con datos registrada (indice base 0): " + ultimaPos);
        System.out.println("Ticket en esa posicion: " + tickets[ultimaPos][0] + " - " + tickets[
    }
}
```

```
<terminated> Ejercicio14_3_6_3_ObtenerUltimaPosicion [Java Application] C:\Program Files\Java\jdk-
=== Ejercicio 14: Seccion 3.6.3 - ObtenerUltimaPosicion ===
Ultima fila con datos registrada (indice base 0): 2
Ticket en esa posicion: 003 - Venta Mostrador 3
```



```java
package a2261330036_estudiodecaso;

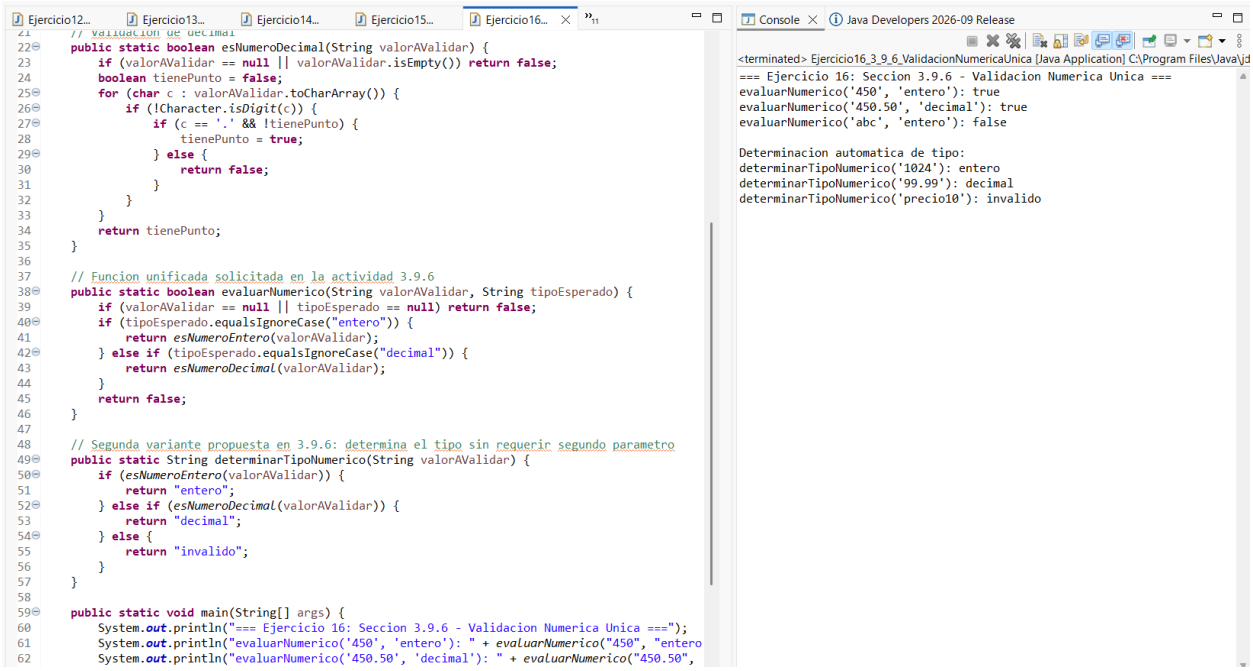
import java.util.Date;

/**
 * Ejercicio 15 - Seccion 3.7: Funcion Fecha (Pagina 95)
 * Descripcion: Retorna la fecha actual del sistema formateada en formato "dd-MM-yyyy"
 * utilizando java.util.Date y java.text.SimpleDateFormat.
 */
public class Ejercicio15_3_7_FuncionFecha {

    // Codigo en Java de la Pagina 95
    public static String Fecha() {
        Date fecha = new Date();
        SimpleDateFormat formatodia = new SimpleDateFormat("dd-MM-yyyy");
        return formatodia.format(fecha);
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 15: Seccion 3.7 - Funcion Fecha ===");
        String fechaHoy = Fecha();
        System.out.println("Fecha actual del sistema: " + fechaHoy);
    }
}
```

```
<terminated> Ejercicio15_3_7_FuncionFecha [Java Application] C:\Program Files\Java\jdk-26.0.2.1\bin\
=== Ejercicio 15: Seccion 3.7 - Funcion Fecha ===
Fecha actual del sistema: 09-11-2026
```



```java
    // Validacion de decimal
    public static boolean esNumeroDecimal(String valorAValidar) {
        if (valorAValidar == null || valorAValidar.isEmpty()) return false;
        boolean tienePunto = false;
        for (char c : valorAValidar.toCharArray()) {
            if (!Character.isDigit(c)) {
                if (c == '.' && !tienePunto) {
                    tienePunto = true;
                } else {
                    return false;
                }
            }
        }
        return tienePunto;
    }

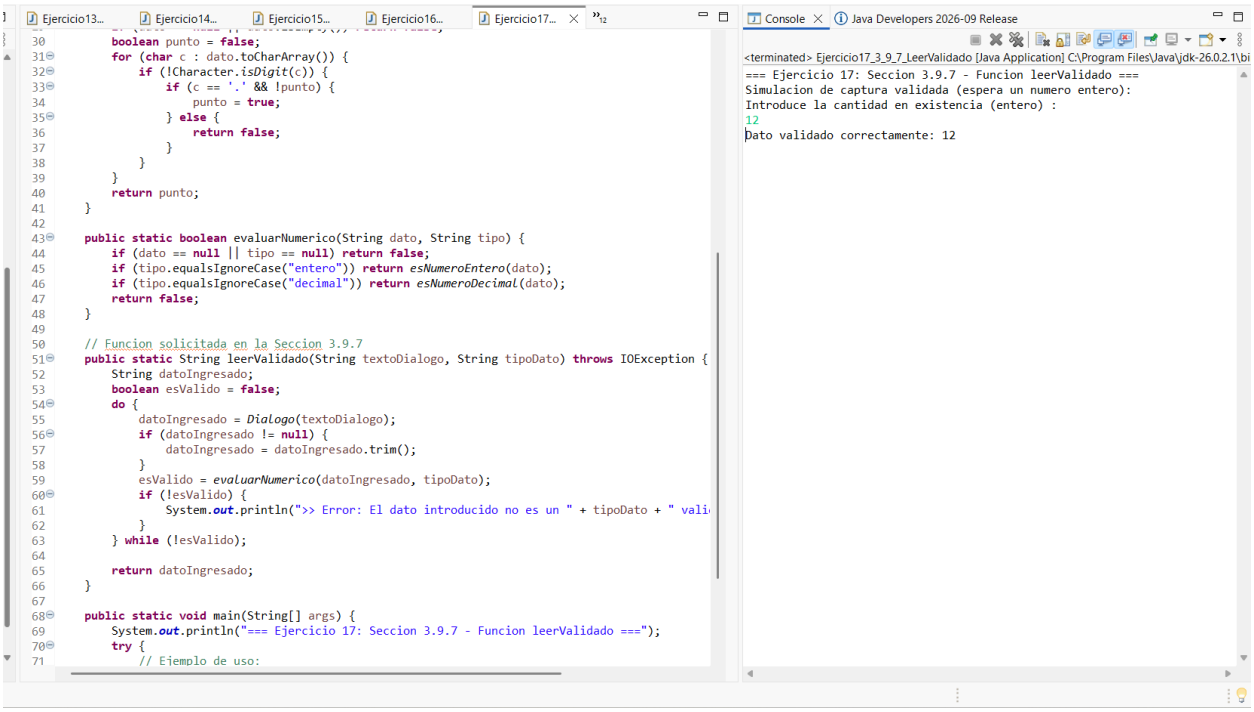
    // Funcion unificada solicitada en la actividad 3.9.6
    public static boolean evaluarNumerico(String valorAValidar, String tipoEsperado) {
        if (valorAValidar == null || tipoEsperado == null) return false;
        if (tipoEsperado.equalsIgnoreCase("entero")) {
            return esNumeroEntero(valorAValidar);
        } else if (tipoEsperado.equalsIgnoreCase("decimal")) {
            return esNumeroDecimal(valorAValidar);
        }
        return false;
    }

    // Segunda variante propuesta en 3.9.6: determina el tipo sin requerir segundo parametro
    public static String determinarTipoNumerico(String valorAValidar) {
        if (esNumeroEntero(valorAValidar)) {
            return "entero";
        } else if (esNumeroDecimal(valorAValidar)) {
            return "decimal";
        } else {
            return "invalido";
        }
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 16: Seccion 3.9.6 - Validacion Numerica Unica ===");
        System.out.println("evaluarNumerico('450', 'entero'): " + evaluarNumerico("450", "entero
        System.out.println("evaluarNumerico('450.50', 'decimal'): " + evaluarNumerico("450.50",
```

```
<terminated> Ejercicio16_3_9_6_ValidacionNumericaUnica [Java Application] C:\Program Files\Java\jd
=== Ejercicio 16: Seccion 3.9.6 - Validacion Numerica Unica ===
evaluarNumerico('450', 'entero'): true
evaluarNumerico('450.50', 'decimal'): true
evaluarNumerico('abc', 'entero'): false

Determinacion automatica de tipo:
determinarTipoNumerico('1024'): entero
determinarTipoNumerico('99.99'): decimal
determinarTipoNumerico('precio10'): invalido
```




```
        boolean punto = false;
        for (char c : dato.toCharArray()) {
            if (!Character.isDigit(c)) {
                if (c == '.' && !punto) {
                    punto = true;
                } else {
                    return false;
                }
            }
        }
        return punto;
    }

    public static boolean evaluarNumerico(String dato, String tipo) {
        if (dato == null || tipo == null) return false;
        if (tipo.equalsIgnoreCase("entero")) return esNumeroEntero(dato);
        if (tipo.equalsIgnoreCase("decimal")) return esNumeroDecimal(dato);
        return false;
    }

    // Funcion solicitada en la Seccion 3.9.7
    public static String leerValidado(String textoDialogo, String tipoDato) throws IOException {
        String datoIngresado;
        boolean esValido = false;
        do {
            datoIngresado = DiaLogo(textoDialogo);
            if (datoIngresado != null) {
                datoIngresado = datoIngresado.trim();
            }
            esValido = evaluarNumerico(datoIngresado, tipoDato);
            if (!esValido) {
                System.out.println(">> Error: El dato introducido no es un " + tipoDato + " vali
            }
        } while (!esValido);

        return datoIngresado;
    }

    public static void main(String[] args) {
        System.out.println("=== Ejercicio 17: Seccion 3.9.7 - Funcion leerValidado ===");
        try {
            // Ejemplo de uso:
```